

How PostgreSQL Replication Actually Works

Explained with desi examples, from the basic idea to the advanced nuance, built around the Tadka case study

Contents

How to read this document

Chapter 1: Why Does a Database Need a "Rough Diary"?

Level 1: The Idea

Level 2: How Tadka Actually Uses It

Level 3: Going Deeper (optional)

Chapter 2: What Exactly Happens If the Power Goes Out Mid-Write?

Level 1: The Idea

Level 2: How Tadka Actually Uses It

Level 3: Going Deeper (optional)

Chapter 3: Copying the Diary to Another Server

Level 1: The Idea

Level 2: How Tadka Actually Uses It

Level 3: Going Deeper (optional)

Chapter 4: Why the Copy Is Always a Little Late

Level 1: The Idea

Level 2: How Tadka Actually Uses It

Level 3: Going Deeper (optional)

Chapter 5: What If the Copy Also Needs Copies?

Level 1: The Idea

Level 2: How Tadka Actually Uses It

Level 3: Going Deeper (optional)

Chapter 6: What Happens When the Manager Suddenly Disappears?

Level 1: The Idea

Level 2: How Tadka Actually Uses It

Level 3: Going Deeper (optional)

Chapter 7: Should Payment Move Out of the House?

Level 1: The Idea

Level 2: How Tadka Actually Uses It

Level 3: Going Deeper (optional)

Chapter 8: Moving Out, Practically

Level 1: The Idea

Level 2: How Tadka Actually Uses It

Level 3: Going Deeper (optional)

Chapter 9: Testing If the New House Actually Protects You

Level 1: The Idea

Level 2: How Tadka Actually Uses It

Level 3: Going Deeper (optional)

Chapter 10: The Promise You Forgot to Write Down

Level 1: The Idea

Level 2: How Tadka Actually Uses It

Level 3: Going Deeper (optional)

Chapter 11: Five Small Ideas Worth Knowing

11.1 Indexing: the textbook index page

11.2 Pagination: flipping pages versus using a bookmark

11.3 Caching: keeping spices on the kitchen counter

11.4 Sharding: many ration shops instead of one giant one

11.5 Two people editing the same list: locking

Chapter 12: Quick Revision, Questions You Might Get Asked

Chapter 13: The One Sentence Worth Remembering

How to read this document

Every topic here is explained in three levels:

Level 1, The Idea. A simple, everyday example. If you read only this part for every topic, you will still understand the whole picture.

Level 2, How Tadka Actually Uses It. The same idea, now connected to a real food-delivery app's database.

Level 3, Going Deeper (optional). One or two extra sentences for the curious reader, or for someone preparing for an interview. Skip this on a first read if you want, nothing later in the document depends on it.

You do not need to remember any command or configuration file from this document. You need to remember the *idea*. The commands can always be looked up later.

Chapter 1

Why Does a Database Need a "Rough Diary"?

Level 1: The Idea

Think of the local kirana shop owner who gives you goods on udhaar (credit). He does not immediately update his big account book for every single customer. Instead, he keeps a small rough diary on the counter. Every time someone buys on credit, he scribbles it there first: "Ramesh, 50 rupees, atta." Only later, maybe at closing time, does he copy all these entries neatly into his main account book.

Why bother with the rough diary at all? Because if the shop catches fire, or the shopkeeper falls asleep mid-way, that rough diary tells him exactly what happened up to that point. Without it, if the account book itself got only half-updated, he would have no way to know which credits were recorded and which were lost.

Level 2: How Tadka Actually Uses It

PostgreSQL, the database behind Tadka, has exactly this rough diary. It is called the **Write-Ahead Log**, or **WAL** for short.

When a customer's order status changes to PAID, PostgreSQL does not directly and only change the main table. It first writes down "order 101 became PAID" in the WAL. Only after that is safely written does it go update the actual table. If the server crashes at the worst possible moment, right between "order in memory says PAID" and "this is safely on disk everywhere," PostgreSQL can look at the WAL on restart and redo whatever did not finish. This is why a PostgreSQL crash almost never loses a committed order.

```
Application says: mark order 101 as PAID
      |
      v
Write it in the WAL first  <-- the rough diary
      |
      v
Then update the actual table
```

Level 3: Going Deeper (optional)

The WAL is not one giant file. It is split into 16MB pieces called segments, and every position inside it has an address called an LSN (think of it as a page number and line number combined). PostgreSQL also periodically takes a "everything up to here is safely saved" snapshot called a checkpoint, so that after a crash it only has to re-read the diary from the last checkpoint onward, not from the very beginning of time.

Chapter 2

What Exactly Happens If the Power Goes Out Mid-Write?

Level 1: The Idea

Think of a registered post office counter. You hand the clerk a letter. The clerk opens the day's dispatch register, writes your entry (name, address, date), and once that entry is fully and neatly written, he stamps a receipt and hands it to you, saying "ho gaya, book ho gaya" (done, it's booked). That stamped receipt in your hand is a promise: no matter what happens to the letter afterward, the post office is now on the hook for it.

Now ask three questions about a power cut at the post office.

What if the power goes out while the clerk is mid-way through writing your entry? The line in the register looks half-finished, maybe just "Ramesh, 12 M" and nothing more, clearly incomplete. Nobody, not even the post office, would trust that scribble as a real, valid entry. It gets ignored, as if it never happened. You never got your receipt anyway, so nothing was promised to you.

What if the entry was fully and neatly written, but the power went out just before the clerk could stamp your receipt? The register technically has your entry, but no promise was ever made to you. From the post office's point of view, this letter was never officially booked. You would have to hand over the letter again.

What if the entry was fully written AND the receipt was stamped and handed to you, but the actual parcel was still sitting in a bin, not yet sent to the sorting godown? This is the important one. Even if the godown catches fire that very night, the post office is still responsible for your letter, because the register (with your fully written, confirmed entry) proves the booking happened. They can reconstruct and resend it from that register entry alone.

Level 2: How Tadka Actually Uses It

This maps almost exactly onto how PostgreSQL treats a crash. Say the app runs:

```
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
```

PostgreSQL's actual sequence is: generate the WAL record for this change, write it into the WAL, flush the WAL to disk, then write and flush a special **COMMIT record** in the WAL, and only then tell the application "done, saved." The actual data page (the real row in the real table) can be updated later, whenever PostgreSQL finds it convenient.

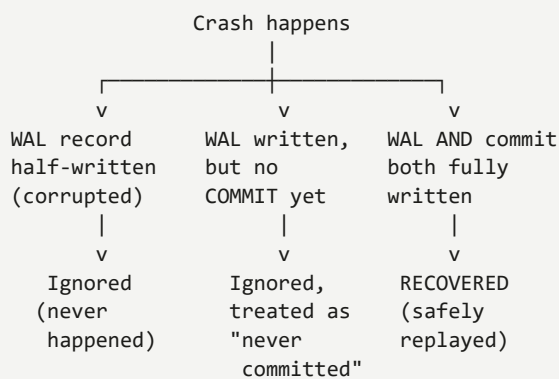
```

Change happens
|
v
WAL record generated
|
v
WAL flushed to disk
|
v
COMMIT record flushed to disk    <-- this is the "stamped receipt"
|
v
Application told "saved successfully"
|
v
(later, whenever convenient) the actual data page is updated

```

Now the three power-cut cases from Level 1, in PostgreSQL's own words:

- **The WAL record itself was left half-written when the crash happened.** On restart, PostgreSQL reads forward through the WAL and simply stops the moment it finds a record that looks incomplete or broken. Everything from that point onward is treated as if it never happened. This is exactly the "half-finished scribble" case, ignored, not trusted.
- **The WAL record was fully written, but the COMMIT record never got flushed before the crash.** PostgreSQL will not recover this transaction. As far as anyone is concerned, it never happened, the same way you never got a stamped receipt, so the post office owes you nothing.
- **Both the WAL record and the COMMIT record were fully flushed, but the actual data page had not been written yet.** PostgreSQL *will* recover this transaction on restart, replaying the WAL to reconstruct the change, exactly the way the post office can still honour your stamped receipt even if the parcel itself was not yet dispatched.



The one-line rule this whole chapter is really about: the diary entry, and the promise stamped on top of it, must be completely safe on disk *before* the actual table is trusted to reflect that change. Never the other way around. If PostgreSQL updated the real table first and only wrote the WAL afterward, a crash in between would leave a changed table with no record explaining why, which is a much harder mess to recover from, or even notice.

Level 3: Going Deeper (optional)

How does PostgreSQL actually tell a genuine, complete WAL entry apart from a half-written, corrupted one? Every WAL record carries its own internal check value (a bit like the post office clerk's habit of always ending an entry with a specific closing mark and a running total). During crash recovery, PostgreSQL reads the WAL forward from the last checkpoint and the moment a record fails that check, it stops trusting anything from that point on, even if there happen to be more bytes physically sitting on the disk after it. It only ever replays the clean, verified prefix of the log, never a guess about what the rest might have meant.

And this is exactly the detail that connects back to replication. The primary's promise, "this WAL and its commit are safely flushed here," is a completely different guarantee from "the replica has already received this WAL." A transaction can be 100% safe on the primary the instant it is committed, while the replica genuinely has not heard about it yet. That gap is precisely what the next chapter is about.

Chapter 3

Copying the Diary to Another Server

Level 1: The Idea

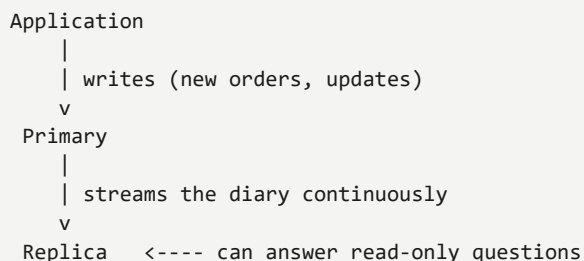
Picture a big wedding catering setup. The head halwai (cook) is preparing a huge pot of sabzi, and a helper stands right next to him, copying every single step into his own pot at the same time, same ingredients, same order, same quantity. If the head halwai suddenly gets a phone call and has to step away, the helper can take over immediately, because he already knows exactly what has been cooked so far.

This helper is not there to cook something different. He is there so that if the head cook is unavailable, someone already knows the current state and can serve the guests without delay.

Level 2: How Tadka Actually Uses It

If the rough diary (WAL) is a complete record of every change made to the database, there is no reason it has to stay on one machine. PostgreSQL can stream that same diary to a second server, which copies every entry and applies it, arriving at the same data, continuously, a few moments behind.

The original server is called the **primary**. It is the only one allowed to accept writes (new orders, updates, deletes). The second server is called a **replica**, or standby. It only copies what the primary tells it and, importantly, it can also answer read-only questions, like "show me this customer's past orders," so that the primary is not the only server doing all the work.



One important rule: you cannot send a new order to the replica. It is a copy that reads instructions, not a place that accepts new instructions.

Level 3: Going Deeper (optional)

This is called **streaming replication**. Technically, the primary runs a small helper process for each connected replica (a WAL sender), and the replica runs a matching process (a WAL receiver) that pulls the diary in real time over the network.

Chapter 4

Why the Copy Is Always a Little Late

Level 1: The Idea

You order food on Swiggy and the live tracking map shows the delivery partner. That dot on the map is never showing you the delivery partner's exact real-time position, it is showing you where the app's server last heard the partner was, maybe five or ten seconds ago. There is always a small delay between "what is actually happening" and "what you are being shown."

Level 2: How Tadka Actually Uses It

The same gap exists between the primary and the replica. Copying the diary across the network and applying it takes real time, even if it is a fraction of a second. This gap is called **replication lag**.

Here is the practical trap: suppose a customer places an order, it is saved on the primary, and one second later the app reads that order back, but the read happens to go to the replica. If the replica has not yet caught up, the app might say "order not found," even though the order genuinely exists, just not on this particular copy yet.

```
Customer places order --> saved on Primary immediately
Customer's app reads it --> goes to Replica
                           |
                           v
                        Replica hasn't caught up yet
                           |
                           v
                    "Order not found" (confusing!)
```

This is not a bug. It is simply what "the replica is a few moments behind" means in practice, and Tadka's design has to account for it.

Level 3: Going Deeper (optional)

The simplest fix is a rule: any read that happens *immediately after* a write by the same customer goes to the primary, not the replica, just for that one specific read. Everything else, browsing the menu, looking at old order history, can safely go to the replica. PostgreSQL can also be configured to make the primary *wait* for the replica to confirm before saying "order saved," closing this gap entirely, at the cost of every order taking a little longer to save.

Chapter 5

What If the Copy Also Needs Copies?

Level 1: The Idea

Imagine a school assembly where the principal has an announcement. If there are only two classes, the principal can walk to each class and say it directly. But if there are fifty classes, the principal cannot personally visit every single one. Instead, the principal tells it to a few class monitors, and each monitor passes it further down to a couple of other classes near them.

This way, the principal only has to repeat the announcement a handful of times, not fifty times, even though eventually every class gets it.

Level 2: How Tadka Actually Uses It

If Tadka needed fifty replicas (it does not, today, but imagine a much bigger version of Tadka), having the primary directly serve all fifty would put a lot of load on it, just for the job of copying the diary out to everyone.

Instead, the primary can send the diary to a few replicas, and those replicas can pass it onward to further replicas behind them. This is called **cascading replication**, and PostgreSQL supports it natively.

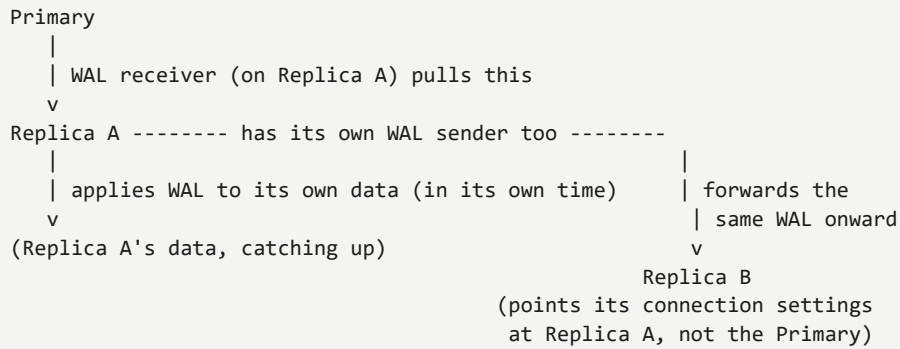
```
Direct (everyone asks the principal directly):
Primary --> R1, R2, R3, R4 ... R50    (a lot of load on Primary)

Cascading (monitors pass it along):
Primary --> R1 --> R3, R4
          --> R2 --> R5, R6
```

The catch: if a class monitor is absent that day, the classes depending on that monitor do not get the announcement either, even if the principal is perfectly fine. Similarly, if an intermediate replica goes down, every replica depending on it stops getting updates too, even though the primary is completely healthy.

How the "class monitor" actually works in PostgreSQL. The interesting bit is that R1, the class monitor in our story, does not need to have fully understood and memorised the announcement before passing it on. In PostgreSQL terms: an intermediate replica does not have to wait until it has finished *replaying* (applying) a piece of WAL to its own data before it is allowed to forward that same WAL further downstream. It only needs to have safely *received and written* that WAL to its own disk. The moment it has that, it can already start handing it to R3 and R4, even while it is still, separately, catching up its own data.

This is why the mechanism is described so simply: every replica already runs the exact same two jobs a primary runs, a receiver that pulls WAL from whoever is upstream of it, and a sender that can hand WAL to whoever is downstream of it. A cascading replica is just a normal replica that happens to also have someone connected downstream. Setting one up is, in practice, the same as setting up any other replica, except the downstream replica's connection settings point at the intermediate replica's address instead of the original primary's address.



Level 3: Going Deeper (optional)

Tadka today has exactly one replica. Building a three-level cascading setup now would only add this "monitor is absent" risk without solving any actual problem, since there is no fan-out problem to solve yet. This is a pattern to know about and reach for later, once there are genuinely many replicas, not something to build in advance just because it exists.

Chapter 6

What Happens When the Manager Suddenly Disappears?

Level 1: The Idea

A branch manager of a bank suddenly resigns without notice. The assistant manager has been watching everything and knows the daily operations well. But does the assistant manager *automatically* become the new manager the moment the old one leaves? No. Someone, head office, has to formally decide "yes, promote the assistant," update the branch nameplate, and inform every customer and every other branch that transactions should now go to this person.

If nobody does this formally, customers might keep trying to reach the old manager, calls might go unanswered, and worse, if the old manager suddenly comes back and does not know they were replaced, you could end up with two people both thinking they run the branch.

Level 2: How Tadka Actually Uses It

A replica having a full, up-to-date copy of the data does not mean it automatically becomes the new primary if the original primary fails. Someone (usually a dedicated monitoring tool, not a human at 3 a.m.) has to detect the failure, promote the replica, and make sure every application server now sends its writes to the new primary instead of the old address.

The dangerous case: if the old primary was not actually dead, just briefly unreachable over the network, and it comes back online still believing it is the primary, you can end up with two servers both accepting writes at the same time. This is called **split-brain**, and it is a serious problem because the two servers' data can drift apart in ways that are very hard to fix afterward.

```
Old Primary (network hiccup, not actually dead)
|
+--> still quietly accepting writes

New Primary (promoted while the old one seemed dead)
|
+--> also accepting writes

Both think they are in charge. Data starts to conflict.
```

Level 3: Going Deeper (optional)

Production PostgreSQL setups use dedicated tools (Patroni is the most common one today) whose entire job is exactly this: watch the primary, agree carefully on whether it has truly failed, promote a replica, and make absolutely sure the old primary can no longer accept writes before trusting the new one, a step called "fencing." A managed database service (like Azure's PostgreSQL offering) usually handles this promotion machinery for you, but you still need to know it is happening and what it guarantees.

Chapter 7

Should Payment Move Out of the House?

Level 1: The Idea

Imagine a joint family living in one big house. Everyone shares the kitchen, the electricity meter, the main door. One family member, let's say the one who runs a small jewellery business from home, starts wondering: "should I move my gold and cash into a separate bank locker instead of keeping it in my room?"

Now, if he moves out just because "everyone in the neighbourhood is buying separate flats these days," that is not a real reason. But if he has an actual reason, his business genuinely needs tighter security than the rest of the house provides, or if a kitchen fire in the shared house could put his gold at risk too, that is a real, defensible reason to want a separate, better-protected space.

Level 2: How Tadka Actually Uses It

Tadka's Payment logic currently lives inside the same application as everything else (Orders, Restaurants), but it already has its own clearly marked "room": its own code folder, its own rules, nobody else's code touches it directly. The question is whether that room should become an actual separate house (a separate service, running as its own independent program).

Weak reasons to move out (heard often, rarely justified on their own):

- *"Payment is slow."* This was already fixed without moving anywhere: Tadka gave the payment call a timeout, capped how many payment calls can run at once, and moved payment processing to the background so customers do not have to wait for it. The slowness problem did not need a house move.
- *"Microservices are the modern way to build things."* This is a fashion statement, not a reason. A cleanly organized room inside the same house already gives most of the benefit, without the cost of moving.
- *"We might want to scale Payment separately someday."* Maybe, but "someday" is not evidence. If Payment genuinely receives ten times the traffic Orders does, that is real evidence. If both receive roughly similar traffic, there is nothing to scale independently yet.

Real reasons to move out:

- *"A crash inside Payment should not bring down the whole house."* This is the strongest reason. If Payment and Orders share one process and Payment's code crashes badly, it can take Orders down with it too, even though Orders had nothing to do with the problem. A separate process genuinely protects against this, the same way a fire in one house does not burn down the neighbour's house.
- *"Payment handles sensitive information (card details, gateway credentials) that deserves stricter access control."* A separate service with its own database can meaningfully shrink who can touch that sensitive data. It is not automatically a compliance certificate by itself, but it is a real, useful boundary.
- *"A different team owns Payment and needs to deploy changes on its own schedule."* If this is actually true (a real team, a real different release rhythm), it is a legitimate reason. If it is the same team deploying everything together anyway, it is not yet a reason.

Level 3: Going Deeper (optional)

Before deciding, ask four questions every time, for any proposed service split, not just Payment: What problem are we actually solving? Does that problem exist *today*, with evidence, or only in theory? Can we solve it without moving out at all? And what new problems will moving out create? A function call inside one process becomes a network call between two processes the moment you split them, and a network call can fail in many more ways than a function call ever can: it can time out, get refused, get lost, or arrive twice.

Chapter 8

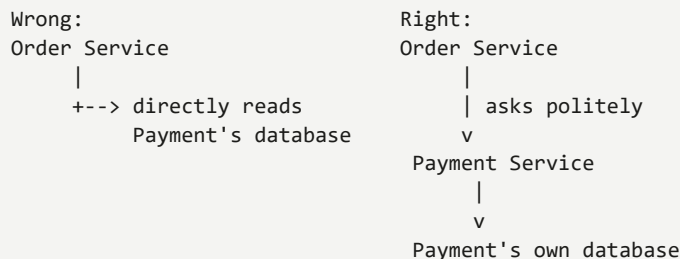
Moving Out, Practically

Level 1: The Idea

When a family member actually moves to a separate flat, a good practice is: do not just carry things in whatever bags are lying around and dump them wherever. Decide clearly what belongs to the new flat (his clothes, his cupboard, his own ration) and what stays in the old house. And when he needs something from the old house, he should call and ask, not walk in and take it from the old cupboard whenever he feels like it.

Level 2: How Tadka Actually Uses It

When Payment becomes its own service, it must **own** its own logic, its own database, and its own gateway integration completely. Orders should never directly read or write Payment's database tables. If Orders needs to know something about a payment, it must ask Payment through a proper request, the same way the family member calls ahead instead of walking into the old house.



Tadka's code already does this the right way: business logic talks to a small interface called `IPaymentClient`, and only one small piece of code actually knows it is making an HTTP call underneath. If tomorrow the way payment is contacted changes, only that one small piece needs to change.

Level 3: Going Deeper (optional)

Once Payment is a separate service, there are now *two* network calls in the chain instead of one: Order calls Payment, and Payment calls the external gateway. Each of these needs its own timeout, sized for what it is actually protecting, not copy-pasted from the other one. And because Orders and Payment now have separate databases, the old trick of "just wrap it in one database transaction" no longer works, a topic Chapter 10 comes back to.

Chapter 9

Testing If the New House Actually Protects You

Level 1: The Idea

You cannot just claim your new flat's door lock is strong. You test it, you actually try to push the door, jiggle the lock, see if it holds. Claiming something is safe and proving it are two different things.

Level 2: How Tadka Actually Uses It

After splitting Payment into its own service, the only real way to know fault isolation actually works is to deliberately switch Payment off and see what happens.

- **Can customers still see the menu?** They should, menu browsing has nothing to do with Payment.
- **Can customers still see their past orders?** They should, that information already lives fully in Order's own database.
- **Can a customer still place a new order?** They should still get a successful "order created" response, with payment marked as pending, to be completed once Payment comes back online.

```
Payment Service: switched off
```

```
See menu?          -> still works
```

```
See old orders?    -> still works
```

```
Place a new order? -> still works, payment marked "pending"
```

If any of these three break the moment Payment goes down, the "separate house" only looks separate on a diagram. Something is still secretly tying them together.

Level 3: Going Deeper (optional)

Common mistakes that quietly break this: letting both services share one database (so a crash in one's queries can still lock up the other), routing every request through a health check that pings every other service first, or having a long synchronous chain (Order waits for Payment, which waits for Notification, which waits for Delivery), so one slow link drags the whole chain down with it.

Chapter 10

The Promise You Forgot to Write Down

Level 1: The Idea

You tell your friend, "I'll transfer you the ₹500 I owe you by evening," but you only remember this as a mental note, nothing written down anywhere. In the middle of the day, your phone's memory gets wiped (imagine a factory reset by accident). By evening, you have completely forgotten about the ₹500. Your friend is still waiting.

If instead you had written it down in a notebook, or set a calendar reminder, that reminder would have survived even though your phone's temporary memory did not.

Level 2: How Tadka Actually Uses It

When Payment is down and a customer places an order, Tadka creates the order immediately and plans to attempt payment a little later, in the background. Right now, that "attempt payment later" plan is only remembered in the running program's memory, not written down anywhere durable.

If the background worker's process restarts for any reason, a deploy, a crash, even a routine update, before it managed to actually process that payment, the reminder is gone. The order still says "payment pending," but nothing is left in the system that remembers it needs to try again.

```
Order created
  |
  v
"Try payment later" reminder, kept only in memory
  |
  X   process restarts
  |
  v
Reminder is gone. Order stays "pending" forever, silently.
```

This is a real, known gap, and the honest thing to do is name it clearly rather than pretend it is solved.

Level 3: Going Deeper (optional)

The proper fix is writing that reminder into the database itself, in the very same transaction that creates the order, so the reminder is exactly as durable as the order. Later, this can grow into a full message queue if the volume becomes large enough to need one, but a simple database table is often the right first step, not a big new piece of infrastructure adopted before it is actually needed.

There is a second, related trap. Suppose the payment gateway *did* successfully charge the customer, but the response got lost before Tadka's server recorded it as successful. If Tadka blindly retries, the customer can be charged twice, the exact fear behind every "paisa kat gaya, order fail ho gaya" (money got deducted, but the order failed) complaint anyone has ever raised on a food app. The fix is attaching a unique reference number to every payment attempt, so a retry using the same reference number is recognised as "the same request, already handled" rather than treated as a brand-new charge.

Chapter 11

Five Small Ideas Worth Knowing

Each of these is a full topic in bigger textbooks. Here is the one-paragraph version of each, enough to hold a sensible conversation about it.

11.1 Indexing: the textbook index page

At the back of a school textbook, there is an index telling you exactly which page has "Photosynthesis." Without it, you would flip through every single page. A database index does the same thing for a table: instead of scanning all one hundred million orders to find one customer's orders, PostgreSQL jumps straight there. The catch: every index you add makes every new entry (every insert) a little slower to write, because the index also has to be updated. So you add indexes for the questions you *actually* ask often, not for every possible question.

11.2 Pagination: flipping pages versus using a bookmark

If you want to read page 500 of a phone book, you would not want to start from page 1 and flip through every page in front of you every single time, that gets slower the further in you go. It is much faster to place a bookmark at your last read position and simply continue from there next time. Databases have the exact same choice: "count from the start every time" (slow for deep pages) versus "remember where I left off" (fast no matter how deep you go).

11.3 Caching: keeping spices on the kitchen counter

You do not walk to the store room every single time you need turmeric while cooking. You keep the commonly used spices right there on the counter. A cache does the same job for data that is read very often but changes rarely, like a restaurant's menu. The real risk is not the cache existing, it is what happens if everyone reaches for the same jar at the exact same moment it runs empty (a "cache stampede"), suddenly sending everyone straight to the store room (the database) all at once.

11.4 Sharding: many ration shops instead of one giant one

Instead of one giant ration shop trying to serve an entire city, the government sets up separate shops per locality. Each shop handles a smaller, manageable crowd. A database can be split the same way, by customer, by region, by whatever key makes sense, so no single server has to hold or serve everything. The trap: if one specific housing society happens to be enormous, all its residents still go to the same one shop, and that one shop gets crowded even though every other shop looks perfectly fine on paper.

11.5 Two people editing the same list: locking

Imagine a shared fridge shopping list. Two family members both look at it, both see "milk" is not crossed out, and both independently decide to buy milk, resulting in two liters when only one was needed. A database avoids this by either checking "has anyone changed this since I last looked?" before saving (this is called optimistic locking), or by putting a temporary lock on the item so only one person can

update it at a time (pessimistic locking). And if two people are trying to update two shared items in a different order from each other, each one holding what the other one wants, that is a **deadlock**, exactly like two people stuck facing each other in a narrow galli, each waiting for the other to move first, and nobody moves.

Chapter 12

Quick Revision, Questions You Might Get Asked

Q1. If a database has a replica, is it automatically safe from downtime? No. A replica is only a copy. Nobody has decided *who* takes over, or *when*, unless a separate mechanism exists specifically for that decision. A copy is not the same thing as a plan.

Q2. A customer just placed an order and immediately can't see it. Why? The read probably went to a replica that has not caught up yet (Chapter 4). The fix is sending that one specific read to the primary, not sending every read there.

Q3. Why not just extract every feature into its own service, since Payment did it? Because "it worked for Payment" is not evidence that Orders, or Restaurants, or anything else has the same real problem. Ask the same four questions from Chapter 7 fresh, every single time.

Q4. Payment succeeded at the gateway, but our server never recorded it. Should we retry? Not blindly. Retrying without a unique reference number for that specific payment attempt can charge the customer twice. Use a reference number so a retry is recognised as "same request," not "new charge."

Q5. Our database's CPU is at 85%. Should we add a read replica? Not immediately. First find out what is actually eating the CPU, a missing index, a bad query, too many open connections. Adding a replica before diagnosing this just means the same badly written query now runs slow on two machines instead of one.

Q6. The menu API gets 20,000 requests a second, but the menu barely changes. What would you do? Cache it. A short expiry time, refreshed on menu updates, is enough. Just make sure not every cached item expires at the exact same second, or you will get a stampede (Chapter 11.3).

Q7. We split Payment into its own service. Does that automatically make the system "more reliable"? Reliable in which specific way? It genuinely improves fault isolation (a Payment crash no longer takes Orders down). It does not automatically make the background payment work durable, that is a separate problem, covered in Chapter 10, and it has to be solved on its own.

Q8. Should Order Service and Payment Service share one database, at least for now, to keep things simple? No. The moment they share a database, nothing actually stops Order from directly touching Payment's tables, and the "separate service" boundary becomes fake. Keep the data separated even if it is inconvenient at first.

Chapter 13

The One Sentence Worth Remembering

A good architect is not the person who knows the most tools. A good architect is the person who can explain, in plain words, why a decision was made, what it cost, and what happens on the day it turns out to be wrong.

Every idea in this document, the rough diary that protects against crashes, the copy that is always a little behind, the family member who moves out for a real reason and not a fashionable one, the promise that has to be written down somewhere durable, exists to make that one sentence something you can actually do in a real conversation, not just something you can repeat.